

ALGORITMO GENÉTICO COM CHAVES ALEATÓRIAS

DCE770 - Heurísticas e Metaheurísticas

Atualizado em: 7 de outubro de 2025

Iago Carvalho

Departamento de Ciência da Computação



Na aula passada vimos que um algoritmo genético pode ser aplicado a, basicamente, qualquer problema de otimização

Apesar de simples e poderoso, sua implementação tem que corresponder exatamente ao problema que está a ser tratado

- Operadores de cruzamento e mutação específicos
- Operadores de reparo de solução

Desta forma, diversos esforços foram desenvolvidos para criar um algoritmo genético mais facilmente adaptável

ALGORITMO GENÉTICO DE CHAVES ALEATÓRIAS

O algoritmo genético de chaves aleatórias (RKGA, do inglês *Random Keys Genetic Algorithm*) foi criado com a proposta de ser um algoritmo genético padrão

Ele possui dois componentes

1. Componente independente do problema: Algoritmo genético
2. Componente dependente do problema: Decodificador

A principal vantagem do RKGA é de focar todo o desenvolvimento algorítmico no componente independente

- Para adaptar o algoritmo a outros problemas, basta algarer o componente dependente

COMPONENTE INDEPENDENTE DO PROBLEMA

O componente dependente é um algoritmo genético como vimos na aula passada

Algoritmo 1: Algoritmo genético

Entrada: Conjunto de parâmetros // A ser definido

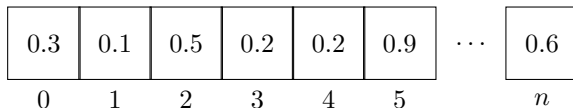
- 1 Inicialização da população
 - 2 Avaliação da população inicial
 - 3 **enquanto** *critério de parada não atingido* **faça**
 - 4 Seleção dos pais
 - 5 Cruzamento
 - 6 Mutação
 - 7 Avaliação das novas soluções geradas
 - 8 Geração da nova população
-

COMPONENTE DEPENDENTE DO PROBLEMA

O componente dependente é baseado em um vetor de chaves aleatórias e um decodificador

O vetor de chaves aleatórias, em geral, é um vetor de variáveis reais com n posições

- Cada posição representa um elemento da solução



Já o decodificador é um algoritmo que decodifica (ou transforma) o vetor de chaves aleatórias em uma solução válida

Esta operação de decodificação é dependente do problema

- Espera-se que seja de baixa complexidade computacional

Veremos alguns exemplos de decodificadores

DECODER PARA O CAIXEIRO VIAJANTE

Seja a um vetor de chaves aleatórias que representa uma solução para um problema do caixeiro viajante com 5 cidades

$$a = \langle \underset{A}{0.25}, \underset{B}{0.19}, \underset{C}{0.67}, \underset{D}{0.05}, \underset{E}{0.89} \rangle$$

Como podemos transformar este vetor em uma solução para o problema do caixeiro viajante?

DECODER PARA O CAIXEIRO VIAJANTE

Seja a um vetor de chaves aleatórias que representa uma solução para um problema do caixeiro viajante com 5 cidades

$$a = \langle \underset{A}{0.25}, \underset{B}{0.19}, \underset{C}{0.67}, \underset{D}{0.05}, \underset{E}{0.89} \rangle$$

Como podemos transformar este vetor em uma solução para o problema do caixeiro viajante?

Basta ordenar as chaves (complexidade $\mathcal{O}(n \log n)$)

$$a = \langle \underset{D}{0.05}, \underset{B}{0.19}, \underset{A}{0.25}, \underset{C}{0.67}, \underset{E}{0.89} \rangle$$

Isto nos gera a rota $D \rightarrow B \rightarrow A \rightarrow C \rightarrow E \rightarrow D$

DECODER PARA O PROBLEMA DA MOCHILA BINÁRIA

Considere a mesma chave aleatória a

$$a = \langle \underset{A}{0.25}, \underset{B}{0.19}, \underset{C}{0.67}, \underset{D}{0.05}, \underset{E}{0.89} \rangle$$

Como podemos transformar este vetor em uma solução para o problema da mochila binária?

Existem duas maneiras simples

- Uma envolve ordenação
- A outra não

DECODER PARA O PROBLEMA DA MOCHILA BINÁRIA

Primeiro método: ordenação

$$a = \langle \underset{D}{0.05}, \underset{B}{0.19}, \underset{A}{0.25}, \underset{C}{0.67}, \underset{E}{0.89} \rangle$$

Podemos fazer um *loop* neste vetor ordenado inserindo (em ordem) todos os itens que cabem na mochila

Segundo método: corte

Todos os itens com valor de chave maior que δ são selecionados

Para $\delta = 0.5$, selecionamos os itens $\{C, E\}$

Este segundo método pode gerar soluções inválidas

- Um mecanismo de reparo é necessário

DECODER PARA O PROBLEMA DA SOMA DE SUBCONJUNTOS

O problema da soma de subconjuntos é um problema de decisão

Dado um conjunto de valores S , devemos selecionar $S' \subset S$ tal que

$$\sum_{s \in S'} x_s = T$$

Considere $T = 1$ e o vetor de chaves aleatórias a como abaixo.
Como você resolveria o problema?

$$a = \langle \underset{A}{0.2}, \underset{B}{0.1}, \underset{C}{0.6}, \underset{D}{0.5}, \underset{E}{0.9} \rangle$$

ALGORITMO GENÉTICO DE CHAVES ALEATÓRIAS TENDENCIOSAS

O RKGA é um algoritmo interessante, mas não é utilizado na prática

Entretanto, seus conceitos são amplamente utilizados pelo Algoritmo Genético de Chaves Aleatórias Tendenciosas

- BRKGA, do inglês *Biased Random Keys Genetic Algorithm*

O BRKGA utiliza os mesmos conceitos de chaves aleatórias e decodificação

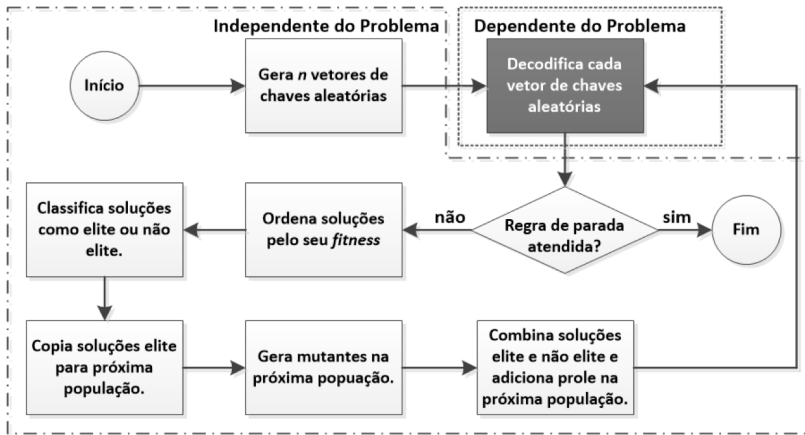
Além disso, ele fornece um *framework* para implementação do algoritmo genético quando associado a estas chaves aleatórias

- Operadores de mutação e cruzamento especializados

FLUXOGRAMA DO BRKGA

O BRKGA pode ser sumarizado pelo seguinte fluxograma

- Soluções elite são aquelas com melhor qualidade de função objetivo



CRUZAMENTO DO BRKGA



CRUZAMENTO DO BRKGA

O cruzamento seleciona dois pais, sendo um da população elite e outro do restante da população

O BRKGA costuma utilizar um cruzamento uniforme como o Algoritmo Genético tradicional

Máscara

0	0	1	0	0	1	1	1	0	0	1	0	1	1	0	1	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



PARÂMETROS DO BRKGA

Assim como o Algoritmo Genético, a grande dificuldade do BRKGA é trabalhar sua parametrização

Tamanho da população (P): Recomenda-se $P = aN$, onde $a \in \mathbb{R}_{\geq 1}$ e N é o tamanho do cromossomo

Tamanho da população elite (P_e): Recomenda-se que $0,10P \leq P_e \leq 0,25P$

Tamanho da população mutante (P_m): Recomenda-se que $0,1P \leq P_m \leq 0,3P$

Probabilidade de herança do elite (c_e): Recomenda-se que $0,5 < c_e \leq 0,8$

PRÓXIMA AULA:
EVOLUÇÃO DIFERENCIAL
E
ALGORITMOS IMUNO-INSPIRADOS